Machine Learning for Mechanical Engineers at CoEP Session 10: Decision Trees

Session 10: DecTree

Decision Tree

Lets play a game

Game

Remember '20 questions' game?

- Some one holds name of one famous person in mind
- Others need to find out that by asking, at max, 20 questions.
- Questions should be such that the answers to which are only Yes or No.
- No queries, no descriptive answers, nothing else.

Game

What are we doing here, essentially?

- Eliminating subgroups.
- So, for question 'Male or Female', any answer will remove half of the population from our search space.
- Each of these questions are actually splitting the search space one by one.
- Making it narrower and narrower.
- Till we get to one person.

Tree? - Comp Science way

- What is a Tree?
- Comp Science definition?
- What are other Data Structures?
- Diff between Tree and Graph?

Decision Tree Example - Admissions

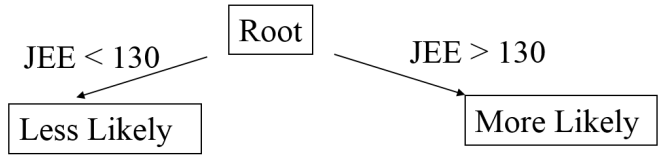
Admissions

- Admission to a reputed college depends, say, on JEE score as well as 12th std marks.
- Previous data of 1000 applicants shows that average JEE score for admitted students is 150 and average 12th std marks are 80%.
- We are asked to create a model that will allow us to predict students most likely to be admitted.
- How do we go about doing this?

Approach

- Divide applicants into subgroups: those with more than, say, like 130 JEE, are in 'more likely' group
- Draw the tree

Approach



Approach

- Split this group itself based 12th percentage, above 70% (called "most likely") or below (called "high maybe").
- Add to the tree

Questions

- What if we split on 12th marks first and then JEE scores - would we get the same groupings?
- What is the best way to split?

Questions

- What if we used more criteria such as essay evaluation scores, extra curricular, awards and distinctions in sports etc. i.e. how many attributes should we use to create splits and what are the most significant attributes.

All these criteria can be modeled using Decision Tree technique.

Decision Tree in Machine Learning

Decision Trees

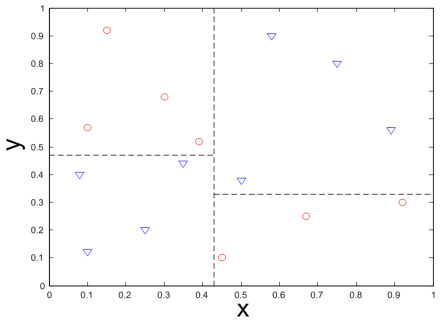
- Decision Trees (DTs) are a non-linear supervised learning method used for classification and regression.
- The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

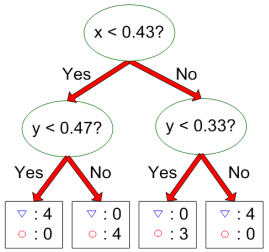
Decision Trees

Can you rewrite the previous decision trees as a If-then-else statement?

Graphically

Decision Boundaries





- Border line between two neighboring regions of different classes is known as decision boundary
- Decision boundary is parallel to axes because test condition involves a single attribute at-a-time

Decision Trees

Simple, yet widely used classification technique

- For categorical target variables. There also are Regression trees, for continuous target variables
- Predictor Features: binary, nominal, ordinal, discrete, continuous.
- Evaluating the model: One metric: error rate in predictions

Decision Trees Structure

- **A root node** has no incoming edges and zero or more outgoing edges
- **Internal nodes**, each of which has exactly one incoming edge and two or more outgoing edges. An internal node is a new question.
- **Leaf/terminal nodes** , each of which has exactly one incoming edge and no outgoing edges. Is a class label. If a leaf node is reached, you got the conclusion.

Decision Trees

- Were really popular 2 decades ago, but fail out of fashion due to over fitting
- Then came newer versions like C4.5, C5 to address some of the issues
- Rather than a single tree, it was found that collection of trees addresses this variance/over-fitting problem
- Thus, now we have Random Forest and Boosting, to make Trees as sought after solution.

Decision Tree for 'Will John Play?'

Predict if John will play tennis

- We have records of outdoors condition and whether John played on those days or not.
- That's the training data.
- We wish to predict for certain UNSEEN condition, whether John will play or not.

Training examples: **9 yes / 5 no**

Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

(Ref: Decision Trees - Victor Lavrenko)

Predict if John will play tennis

Steps:

- Split at columns
- Are they pure? (all yes or all no)
- If yes: stop; if not: repeat

Lets start with first column" "Outlook"

Predict if John will play tennis

Training examples: **9 yes / 5 no**

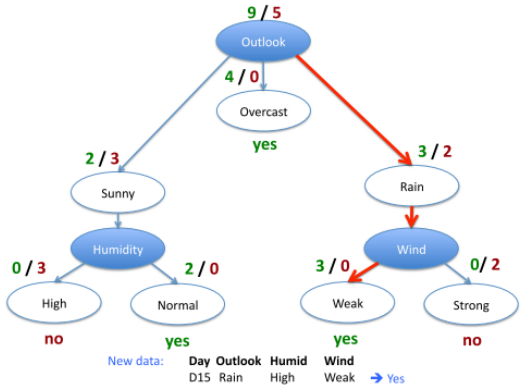
Day	Outlook	Humidity	Wind	Play
D1	Sunny	High	Weak	No
D2	Sunny	High	Strong	No
D3	Overcast	High	Weak	Yes
D4	Rain	High	Weak	Yes
D5	Rain	Normal	Weak	Yes
D6	Rain	Normal	Strong	No
D7	Overcast	Normal	Strong	Yes
D8	Sunny	High	Weak	No
D9	Sunny	Normal	Weak	Yes
D10	Rain	Normal	Weak	Yes
D11	Sunny	Normal	Strong	Yes
D12	Overcast	High	Strong	Yes
D13	Overcast	Normal	Weak	Yes
D14	Rain	High	Strong	No

New data:

D15	Rain	High	Weak	?

Predict if John will play tennis

- Decision tree result can be seen below.
- To test the unseen condition, traverse the tree
- for Day 15, with Outlook-Rain, Humid-High and Wind-Weak, the outcome is "Yes". John will play.



Decision Tree Classifier

- Decision tree models are relatively more descriptive than other types of classifier models
  - Easier interpretation
  - Easier to explain predicted values
- Exponentially many decision trees can be built
- Which is best? Some trees will be more accurate than others
- How to construct the tree? Computationally infeasible to try every possible tree.

Building the Decision Tree

What is the "Best Attribute" to split

There are different criteria that can be used to determine what is the best attribute to split.

- Information Gain
- Gini Index
- ...

Which attribute to split on

- Want to measure PURITY of the split
- Are we more certain about Yes/No after the split?
  - pure set (4 yes/ 0 no) then completely certain (100%)
  - impure set (3 yes/ 3 no) then completely uncertain (50%)
- Must be symmetric: 4 yes / 0 no is just as pure as 0 yes / 4 no.

Entropy

- High 'mixed'-ness, high uncertainty thus higher "entropy"
- Say if all are identical members, then no uncertainty and thus zero "entropy".

Entropy

- So, when we split a set we want the halves to be more distinct from each other and the members in the group to be more like each other
- We want entropy to go down as we keep splitting.
- So if we use an approach/split that doesn't reduce the entropy by much, then it is probably not a good attribute or a good value to split on.
- Shannon's entropy is defined for a system with N possible states as $S = -\sum_{i=1}^{N} p_i \log_2 p_i$ where p_i is the probability of finding the system in the i-th state.
- N designates number of classes. For binary classification $N = 2$.
- Shannon's formula is part of information theory, given minimum number of letters to encode a message.

Entropy

- Originally from Second Law of Thermodynamics, Entropy is a measure of disorder (uncertainty, chaos).
- In the current context, its measure of disorder in the target-label.
- Entropy can be described as the degree of chaos in the system.
- The higher the entropy, the less ordered the system and vice versa.
- Information Gain is a measure of decrease in Entropy by partitioning the data-set.

How to calculate Information Gain using Entropy?

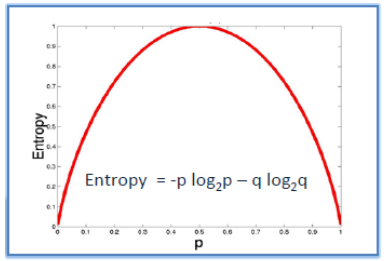
Information Gain

- Want many items in pure sets.
- Try to split on each possible feature in a dataset. See which split works "best".
- Measure the reduction in the overall entropy of a set of instances.

$InformationGain = Entropy(s) - \sum \frac{s_i}{s} E(s_i)$

Entropy

- ID3 algorithm uses entropy to calculate the homogeneity of a sample.
- If the sample is completely homogeneous the entropy is zero
- If the sample is equally divided then it has entropy of one.
- By Binary outcome, $Entropy(n) = -p_{(+)} log_2 p_{(+)} - p_{(-)} log_2 p_{(-)}$
- $Entropy(n) = -0.5 log_2 0.5 - 0.5 log_2 0.5 = 1$

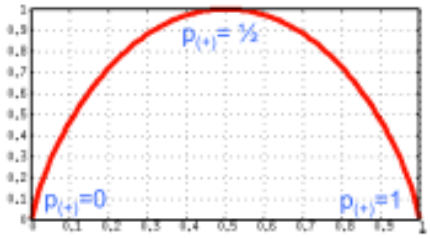


Entropy = -0.5 $\log_2$0.5 – 0.5 $\log_2$0.5 = 1

Entropy

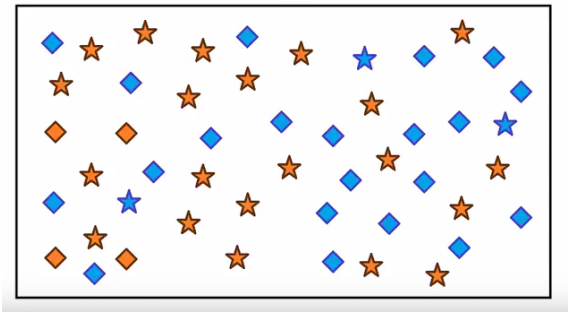
In our original example:

- Impure (3 yes/ 3 no): $Entropy(n) = -3/6 log_2 3/6 - 3/6 log_2 3/6 = 1$
- Pure (4 yes/ 0 no): $Entropy(n) = -4/4 log_2 4/4 - 0/4 log_2 0/4 = 0$
- So, Entropy also ranges from 0 to 1, in following fashion



Information Gain Exercise of Stars/Diamonds, Blue/Orange

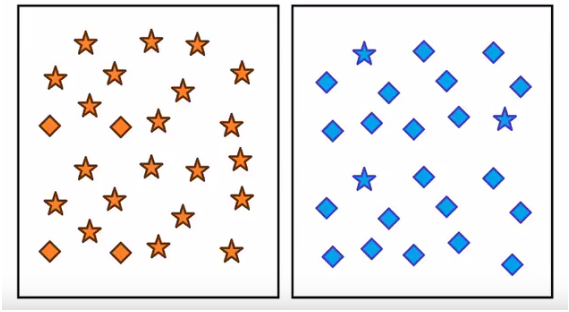
Entropy Example



- There are stars and diamonds, 24 stars, 25 diamonds
- Input: Color
- Output: Shape

Entropy Example

- Predicting: Star vs Diamond
- Split on: Color
- Orange box and blue box.
- With this, HAS it becomes easier to predict Star or Diamond?
- Lesser chance of mis-classification.
- Less disorder, more information gain.



GINI Index

(Ref: Decision Tree. It begins here - Rishabh Jain)

Gini Index

Gini index says, if we select two items from a population at random then they must be of same class and probability for this is 1 if population is pure.

- Works with categorical target variable
- Performs only Binary splits
- Higher the value of Gini higher the homogeneity
- CART (Classification and Regression Tree) uses Gini method to create binary splits

Gini Index Steps

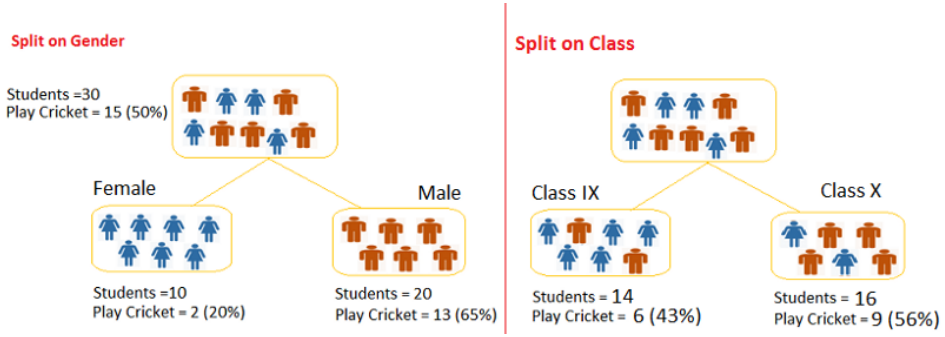
Steps to Calculate Gini for a split

- Calculate Gini for sub-nodes, using formula sum of square of probability for success and failure $(p^2 + q^2)$
- Calculate Gini for split using weighted Gini score of each node of that split

Example

- We want to segregate the students based on target variable (playing cricket or not)
- We split the population using two input variables Gender and Class.
- Now, I want to identify which split is producing more homogeneous sub-nodes using Gini index.

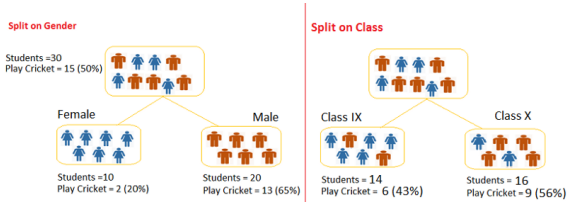
Entropy



(Note:characters shown in the box are not proportional to numbers shown outside!!!)

Split on Gender

- Gini for sub-node Female $= (0.2) * (0.2) + (0.8) * (0.8) = 0.68$
- Gini for sub-node Male $= (0.65) * (0.65) + (0.35) * (0.35) = 0.55$
- Weighted Gini for Split Gender $= (10/30) * 0.68 + (20/30) * 0.55 = 0.59$



Split on Class

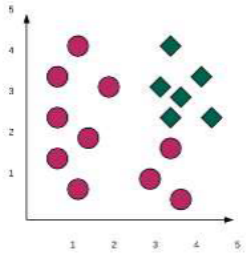
- Gini for sub-node Class IX $= (0.43) * (0.43) + (0.57) * (0.57) = 0.51$
- Gini for sub-node Class X $= (0.56) * (0.56) + (0.44) * (0.44) = 0.51$
- Weighted Gini for Split Class $= (14/30) * 0.51 + (16/30) * 0.51 = 0.51$

Above, you can see that Gini score for Split on Gender is higher than Split on Class, hence, the node split will take place on Gender.

GINI Index (Recap)

A measure of inequality developed by an Italian named Gini.

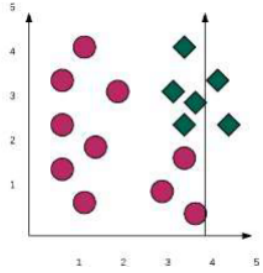
- Definition : Expected error rate $G(S) = 1 - [\sum p(j|t)^2]$
- GINI =0 (All elements are same class)
- GINI =0.5 (All elements evenly split between classes)



$G(t) = 1 - [(\frac{10}{16})^2 + (\frac{6}{16})^2] = 0.46875$

GINI Gain (Recap)

- Definition : $G(A, S) = G(S) - \sum(s_j/s)G(S)$
- If we Split $X < 4$
- $Gini < 4 = 0.4081$
- $Gini > 4 = 0$
- Gini Gain = 0.4687 - 10/16 (0.4081) - 6/16 (0)
- Gini Gain = 0.2136



When to use which?

- Gini for continuous attributes
- Entropy for categorical.
- Entropy is slower to calculate than GINI
- Gini may fail with very small probability
- Difference between the two is theoretically around 2%

Intuition

Both formulas answer the same question – "how mixed is this node?" – just with different math. Entropy's $\log_2$ term is expensive to compute at every candidate split of every node, while Gini's plain p^2 is just a multiplication – cheaper, which is why CART (and scikit-learn's default) uses Gini even though the two rarely disagree on which split is best.

Advantages and Disadvantages of Trees

Advantages

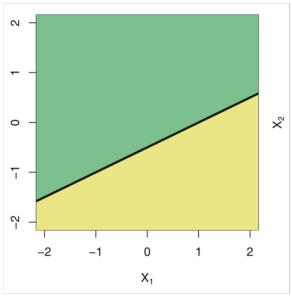
- Trees are very easy to explain
- Easier to explain than linear regression
- Trees can be displayed graphically and interpreted by a non-expert
- Decision trees may more closely mirror human decision-making

Disadvantages

- Trees usually do not have same level of predictive accuracy as other data mining algorithms
- But, predictive performance of decision trees can be improved by aggregating trees. Techniques: bagging, boosting, random forests

Compared to Linear Models

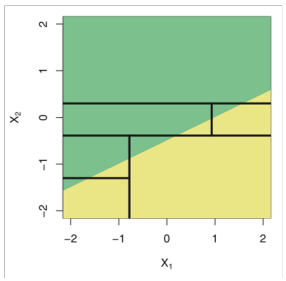
Two-classes: {green, blue}



Linear model can perfectly separate the two regions.

- What about a decision tree?

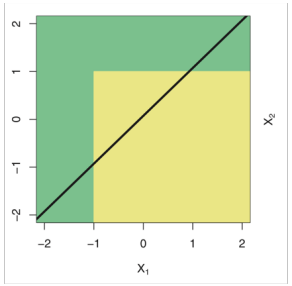
Decision boundary: linear



Decision tree cannot separate regions.

Compared to Linear Models

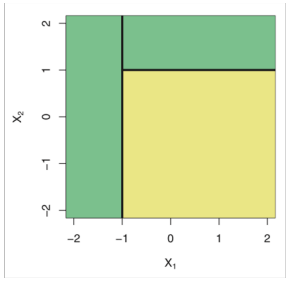
Two-classes: {green, blue}



Linear model cannot perfectly separate the two regions.

- What about a decision tree?

Decision boundary: nonlinear



A Decision tree can!

Decision Trees (Recap)

- A decision tree or a classification tree is a tree in which each internal (non-leaf) node is labeled with an input feature.
- The arcs coming from a node labeled with a feature are labeled with each of the possible values of the feature.
- Each leaf of the tree is labeled with a class or a probability distribution over the classes.
- For instance, in the example below, decision trees learn from data to approximate a sine curve with a set of *If-Then-Else* decision rules.
- The deeper the tree, the more complex the decision rules and the fitter the model.

Pros and Cons of Single Decision Tree

Pros

- Simple interpretations
- Can be built quickly.
- Does built-in Feature selection. So, if any of the features does not get used in Splitting, it is, sort-of, ignored, not selected in prediction.

Cons

- Small changes in data affects structure of tree
- So, instead of single tree, a collection of Trees is preferred

Such collection is called as 'Ensemble'.

Ensemble Methods

- Currently using one single classifier induced from training data as our model, to predict class of test instance
- What if we used multiple decision trees?
- Motivation: committee of experts working together are likely to better solve a problem than a single expert
- But no "group think": each model should make predictions independently of other models in the ensemble
- In practice: methods work surprisingly well, usually greatly improve decision tree accuracy

Quick Check: Decision Trees

Think About It

A single decision tree, grown fully, gets 100% accuracy on its own training data but does much worse on new data. If we build an ensemble of many trees and average their predictions, why does that help – doesn't averaging several overfit models just give you an overfit average?

Answer

It helps as long as the trees' *errors* are not all the same. A single fully-grown tree overfits to the specific noise/quirks of its training sample, and different random samples (or subsets of features) produce trees that overfit to *different* noise. Real signal tends to agree across trees; noise-driven mistakes tend to cancel out when averaged, because they point in different, uncorrelated directions. This is exactly why Random Forest deliberately randomizes both the training rows (bagging) and the candidate features per split – decorrelating the trees' errors is the whole point, not an accident.